Azure DevOps Management Plan

1. Introduction This document outlines how our team will use Azure DevOps to plan, track, and deliver our project. It covers backlog structure, item naming, board setup, sprint routine, and team responsibilities.

2. Backlog Structure

- **Epics**: High-level themes (e.g., Self Study, Project Planning).
- **Issues**: User stories tied to Epics. Write as: "As a [role], I want [action] so that [benefit]."
- **Tasks**: Concrete steps to complete an Issue. Short, action-focused descriptions.

3. Work Item Naming & Tags

Work Items should be named intuitively and so that they are easily recognizable.

- **Tags**: Use labels like sprint1, sprint2, blocker, or feature tags (api, docker) for better overview.

Core Tag Categories

Tag	Use For
`self-study`	Anything involving individual or team learning
`planning`	Work related to roadmaps, backlogs, and project planning
`documentation`	Any document creation, requirement logs, etc.
`demo`	Tasks contributing to preparing or presenting demos
`evidencing`	Evidence creation, template setup, sprint logs
`architecture`	Initial and ongoing architectural decisions
`pipeline`	Any CI/CD, data, or ML pipeline setup
`api`	API creation or updates
`deployment`	Azure ML, Docker, or general deployment-related work
`modeling`	Building or configuring models
`docker`	Docker container setup
`python-package`	Python packaging and project structure
`virtual-env`	Setting up virtual environments for Python
`management`	Anythin related to project, repo, azure devops or team management

Secondary/Contextual Tags

Tag	Use For
`azure-devops`	Tasks directly related to managing work in Azure DevOps
`python`	For any Python-specific work including environments or package management
`github`	Git or GitHub related practices, versioning, and collaboration
`mlops`	When a task connects directly to MLOps workflows

Tag	Use For
`presentation`	Presentation tasks and design
`template`	Making reusable templates (docs, repos, packages)
`sprint-1`	Tasks belonging to Sprint 1 (add later for Sprint 2, 3...)
`team/individual-task`	When all members must perform the task as a team or individually
`initial-setup`	One-time setup tasks for tools, environments, etc.

Example Tag Combinations

Work Item	Suggested Tags
Study Git and GitHub best practices	`self-study`, `github`, `team-task`
Make project plan	`planning`, `documentation`, `azure-devops`
Set up Azure ML Deployment	`deployment`, `azure-devops`, `mlops`

4. Areas & Iterations

- Project root area: CV9.
- Iterations set to sprints:
  - CV9\Sprint 1
  - CV9\Sprint 2
  - ...
  - CV9\Sprint 5

5. Board Setup & Columns We will use a Kanban-style board with these columns:

- **To Do / Project Backlog**: All prioritized Issues for future sprints.
- **Sprint Backlog**: Issues and Tasks committed for the current sprint.
- **In Progress**: Items actively being worked on.
- **Has Bugs / Need Review**: Items that failed tests or need feedback.
- **In Review**: Items ready for code or design review.
- **Done**: Completed items that meet acceptance criteria.

Map work item states to columns in Azure DevOps board settings.

6. Sprint Workflow

- **Sprint Planning**: At start of sprint, move selected Issues from Project Backlog to Sprint Backlog. Break them into Tasks if needed and estimate effort.
- **Daily Stand-up**: 10-15 minutes. Each member answers: What did I do yesterday? What will I do today? Any blockers?

- **Review & Demo**: At sprint end, demonstrate completed work in "Done".
- **Retrospective**: Discuss what went well, what didn't, and improvements for next sprint.

7. Roles & Responsibilities

- **Scrum Master**: Facilitate meetings, remove blockers, maintain board health.
- **Product Owner**: Prioritize backlog, clarify requirements, accept completed work.
- **Team Members**: Create and update work items, estimate effort, complete Tasks, test and document work.

8. Task Member Assignment

- All individual tasks should have an assigned team member.
- Group or multiple member tasks will not be assigned members as that is not possible in our current Azure DevOps Board.

For deciding who to assign the task to, we will first ask for preference, and then assign at random the remaining tasks.

9. Maintenance & Best Practices

- **Daily Backlog Grooming**: Review and refine backlog items as requirements evolve.
- **Work Item Updates**: Keep descriptions, tags, and estimates current.
- **Branch & PR Practices**: Use feature branches, descriptive commit messages, and pull requests with at least one review.
- **Documentation**: Attach relevant links or files to work items (design docs, meeting notes).

10. Item Review Guidelines

When an item is placed in the **"Has Bugs / Need Review"** column, the following process will be used to ensure it is properly reviewed:

Assigning a Reviewer

- A team member can either **volunteer** or be **assigned** to review the item.
- Once a reviewer is confirmed, the item is moved to the **"In Review"** column.

Review Process

The reviewer should:

- Examine the work and check if it meets the **acceptance criteria**.
- Look for any **missing components**, **errors**, or **opportunities for improvement**.
- If everything is in order:
    - Leave a comment such as *"Reviewed – everything looks good."*
    - Move the item to the **"Done"** column.

- Update the work item status as needed.

If Changes Are Needed

If adjustments are required, the reviewer has two options:

1. **Leave a Comment**
    - Clearly describe what needs to be fixed or added.
    - The original assignee or another team member makes the changes.
2. **Make the Changes Directly**
    - Create a new Git branch for the fix or improvement.
    - Push a **pull request** (PR) with the changes.
    - Add a comment on the item noting:
        - A branch was created
        - A short summary of what was changed
    - After the PR is reviewed and merged, move the item to **"Done"**.

This plan will guide our use of Azure DevOps to ensure clarity, accountability, and steady progress throughout the project.

Acknowledgment The structure and content of this repository management plan were partially assisted by ChatGPT to ensure clarity, consistency, and alignment with best practices in GitHub and MLOps documentation.